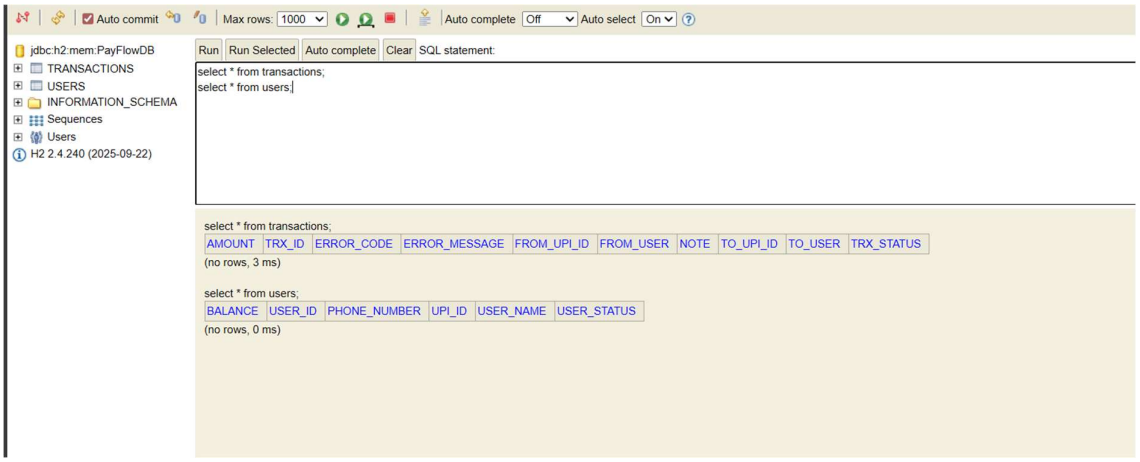
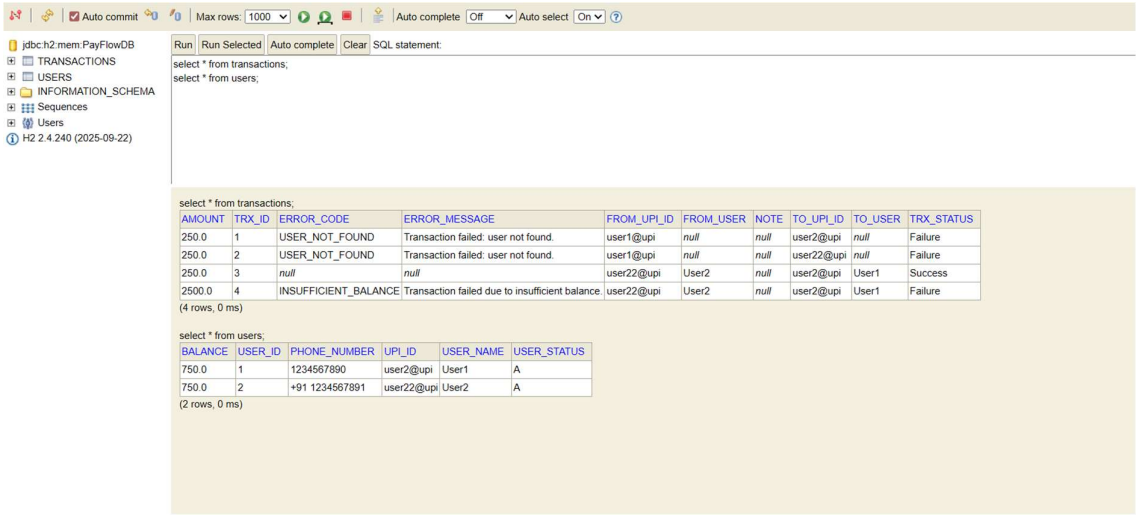
Task 2:



Task 5:

```
C:\Users\manoj>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d "{\"userName\":\"User1\",
\"upiId\":\"user2@upi\",\"balance\":500,\"phoneNumber\":\"1234567890\"}"
{"userId":1,"userName":"User1"}
C:\Users\manoj>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d "{\"userName\":\"User2\",
\"upiId\":\"user22@upi\",\"balance\":1000,\"phoneNumber\":"+91 1234567891"}"
{"timestamp":"2026-06-08T20:13:19.675Z","status":400,"error":"Bad Request","path":"/users"}curl: (3) unmatched close bra
ce/bracket in URL position 11:
1234567891}
          ^

C:\Users\manoj>curl -X POST http://localhost:8080/users -H "Content-Type: application/json" -d "{\"userName\":\"User2\",
\"upiId\":\"user22@upi\",\"balance\":1000,\"phoneNumber\":\"+91 1234567891\"}"
{"userId":2,"userName":"User2"}
C:\Users\manoj>curl -X POST http://localhost:8080/transactions -H "Content-Type: application/json" -d "{\"fromUpiId\":\"
user1@upi\",\"toUpiId\":\"user2@upi\",\"amount\":250.00}"
{"message":"Transaction failed: user not found.","trxId":1}
C:\Users\manoj>curl -X POST http://localhost:8080/transactions -H "Content-Type: application/json" -d "{\"fromUpiId\":\"
user1@upi\",\"toUpiId\":\"user22@upi\",\"amount\":250.00}"
{"message":"Transaction failed: user not found.","trxId":2}
C:\Users\manoj>curl -X POST http://localhost:8080/transactions -H "Content-Type: application/json" -d "{\"fromUpiId\":\"
user22@upi\",\"toUpiId\":\"user2@upi\",\"amount\":250.00}"
{"message":"Transaction has been successfully processed.","trxId":3}
C:\Users\manoj>curl -X POST http://localhost:8080/transactions -H "Content-Type: application/json" -d "{\"fromUpiId\":\"
user22@upi\",\"toUpiId\":\"user2@upi\",\"amount\":2500.00}"
{"message":"Transaction failed due to insufficient balance.","trxId":4}
C:\Users\manoj>
```



Answer to point 4. >> When you send a request to POST /users without @RequestBody, Spring does not read the JSON from the HTTP request body, so the Users object inside the controller is not populated from the client data. Its fields will usually be null or default values because Spring treats it like a model attribute instead of binding the raw JSON. With @RequestBody, Spring converts the JSON payload into the Users object, so the fields like userName, upiId, balance, and phoneNumber contain the values sent by the client.

Task 6:

Answers to point 4.>>

Request lifecycle

When curl sends POST /users, the request first reaches Spring Boot's embedded server, which forwards it into the Spring MVC pipeline. The DispatcherServlet acts like the central front controller: it receives the request and asks Spring to find the right controller method. Then a HandlerAdapter invokes the matched handler method, which in your project is UserController.addUserDetails(...). After that, Spring passes the request body through message conversion, creates the Users object, and finally calls your createUser-style logic in the service layer.

Serialization

For a JSON payload like {"name":"Priya","upiId":"priya@okaxis"}, Spring uses Jackson to convert the JSON into a Java object. Jackson looks at the JSON keys and matches them to Java field names or setter names, so upiId maps cleanly to upiId. If the key is written as upi_id instead, it will not automatically match upiId unless you add an annotation like @JsonProperty("upi_id") or configure a naming strategy. Without that, the upiId field would likely stay null.

Spring Boot features

The three Spring Boot features are embedded server, auto-configuration, and production-ready defaults. In PayFlow, the embedded server is the Tomcat instance that runs your app directly from the JAR, so you do not need to install a separate server. Auto-configuration is doing work when Spring automatically wires your UserRepository, TransactionRepository, H2 datasource, and JSON handling based on the dependencies and annotations. Production-ready defaults show up in things like sensible logging, built-in error responses, connection pooling, and the H2 console support.

Spring vs. Spring Boot

If you used plain Spring, you would have had to configure much more by hand. You would likely need manual XML or Java configuration for component scanning, the dispatcher servlet, JPA setup, datasource wiring, and an external server deployment setup. Spring Boot takes care of most of that automatically using conventions and starter dependencies. That is why your PayFlow app can run with a simple main method and a few properties in application.properties.

Stateless REST

Stateless means each request contains all the information needed to process it, and the server does not rely on memory of previous requests. In your POST /transactions endpoint, the transaction should be processed based only on the current request body and the database state, not on any session data stored in the server. This matters a lot if PayFlow runs on three servers behind a load balancer, because any request can land on any server. If the API were stateful, one server might know something that the others do not, which would break consistency and make scaling unreliable.

Persistence

If you had stored transactions in a Java List and restarted the server, all the records would have disappeared because a List only lives in memory. That means every reboot would erase your transaction history, balances, and audit trail. For a payments app, that is unacceptable because money movement must be durable, traceable, and recoverable even after crashes or restarts. Using H2 gives you persistence through the database layer instead of depending on temporary RAM storage.